École Nationale des Sciences Appliquées d'Al Hoceima
Université Abdelmalek Essaadi

Compte Rendu de Travaux Pratiques

Module : Java EE (JEE)

Application de Gestion des Produits – MVC1

Servlet · JSP · JSTL · Authentification · Rôles

Réalisé par : Nabila Es-Saidy

Encadrant : M. Cherradi

Année : 2024 – 2025

Table des matières

Introduction **3**

1 Présentation générale de l'application **4**
1.1 Objectifs du projet 4
1.2 Fonctionnalités implémentées 4
1.3 Structure du projet 5

2 Architecture MVC1 **6**
2.1 Principe du patron MVC1 6
2.2 Couche Modèle (Model) 6
2.2.1 Entité Produit 6
2.2.2 Entité Utilisateur 7
2.3 Couche DAO 7
2.4 Couche Service 8

3 Sécurité : Authentification et gestion des rôles **10**
3.1 Chaîne de filtres 10
3.1.1 Filtre d'authentification 10
3.1.2 Filtre de rôle 11
3.2 Configuration dans web.xml 12

4 Servlets – Contrôleurs de l'application **13**
4.1 Servlet de connexion – `LoginServlet` 13
4.2 Servlet de liste – `ListProduitServlet` 14
4.3 Servlet d'ajout – `AddProduitServlet` 15
4.4 Servlet de déconnexion – `LogoutServlet` 15

5 Vues JSP **17**
5.1 Page de connexion – `login.jsp` 17
5.2 Page principale – `index.jsp` 18
5.3 Page d'erreur – `erreur.jsp` 19

6 Flux de travail et scénarios d'utilisation **22**

6.1 Vue d'ensemble du flux 22
6.2 Scénario 1 – Connexion réussie (ADMIN) 23
6.3 Scénario 2 – Connexion réussie (USER) 23
6.4 Scénario 3 – Identifiants incorrects 23
6.5 Scénario 4 – Ajout d'un produit (ADMIN) 24
6.6 Scénario 5 – Modification d'un produit (ADMIN) 24
6.7 Scénario 6 – Suppression d'un produit (ADMIN) 25
6.8 Scénario 7 – Tentative d'accès interdit (USER → Erreur 403) 25
6.9 Scénario 8 – URL introuvable (Erreur 404) 25
6.10 Scénario 9 – Déconnexion 26

7 Tests et résultats **27**
7.1 Tests fonctionnels 27
7.2 Captures d'écran des résultats 28

Conclusion **29**

A Comptes de démonstration **30**

B Dépendances Maven **31**

Introduction

Dans le cadre du module **Java EE (JEE)**, ce compte rendu présente la réalisation d'une application web de gestion de produits suivant le patron architectural **MVC1** (*Model–View–Controller*, variante 1). Cette application illustre les concepts fondamentaux du développement web Java EE : les Servlets, les pages JSP, la bibliothèque de balises JSTL, la gestion de session, et la sécurisation des accès par filtres.

L'application offre un système complet comprenant l'authentification des utilisateurs, une gestion des rôles (`ADMIN` et `USER`), les opérations CRUD sur les produits, ainsi qu'une gestion des erreurs HTTP (403, 404, 500).

Ce document décrit l'architecture technique, les composants implémentés, les différents scénarios d'utilisation, et illustre chaque fonctionnalité par des extraits de code commentés et des captures d'écran.

1. Présentation générale de l'application

1.1 Objectifs du projet

L'application *GestionProduitsMVC1* est une application web Java EE développée sans framework (pas de Spring, pas de Hibernate), s'appuyant uniquement sur :

- **Java Servlets** — contrôleurs HTTP
- **JSP / JSTL** — vues dynamiques
- **Filtres Servlet** — sécurité et contrôle d'accès
- **Maven** — gestion des dépendances et du build
- **Apache Tomcat** — serveur d'application

1.2 Fonctionnalités implémentées

Fonctionnalité	Description	Rôle requis
Authentification	Connexion par login/mot de passe avec session	Tous
Déconnexion	Invalidation de la session	Tous
Liste des produits	Affichage de tous les produits	Tous
Recherche par ID	Filtrage de la liste par identifiant	Tous
Ajout de produit	Création d'un nouveau produit	ADMIN
Modification	Édition en ligne d'un produit existant	ADMIN
Suppression	Suppression avec confirmation	ADMIN
Erreur 403	Accès refusé si l'utilisateur n'est pas ADMIN	Tous
Erreur 404	Page introuvable	Tous
Erreur 500	Erreur serveur interne	Tous

Table 1.1 – Récapitulatif des fonctionnalités

1.3 Structure du projet

```
GestionProduitsMVC1/
 pom.xml
 src/main/
     java/
        dao/
           Produit.java          (entité)
           ProduitDAO.java       (interface)
           ProduitImpl.java      (impl. en mémoire)
        models/
           Utilisateur.java      (entité utilisateur)
        services/
           ProduitMetier.java    (interface service)
           ProduitMetierImpl.java(impl. Singleton)
           AuthService.java      (auth. Singleton)
        filters/
           AuthFilter.java       (vérif. session)
           RoleFilter.java       (vérif. rôle)
        web/
            LoginServlet.java
            LogoutServlet.java
            ListProduitServlet.java
            AddProduitServlet.java
            EditProduitServlet.java
            UpdateProduitServlet.java
            DeleteProduitServlet.java
     webapp/
         css/style.css
         index.jsp
         WEB-INF/
             web.xml
             views/
                 login.jsp
                 index.jsp
                 erreur.jsp
```

Figure 1.1 – Arborescence du projet Maven

2. Architecture MVC1

2.1 Principe du patron MVC1

Dans le patron **MVC1**, la vue (JSP) joue un rôle central : elle reçoit directement les requêtes HTTP et délègue la logique métier à des JavaBeans ou services. Ici, les **Servlets** jouent le rôle de contrôleurs qui orchestrent le traitement :

1. Le navigateur envoie une requête HTTP.
2. Un **Filtre** vérifie l'authentification (et le rôle si nécessaire).
3. La **Servlet** (contrôleur) traite la requête, appelle le **Service** métier.
4. Le service appelle la couche **DAO**.
5. Le contrôleur place les données en attributs de requête et *forward* vers la **JSP** (vue).
6. La JSP génère le HTML retourné au navigateur.

2.2 Couche Modèle (Model)

2.2.1 Entité Produit

```java
public class Produit {
    private Long   idProduit;
    private String nom;
    private String description;
    private Double prix;

    public Produit() {}

    public Produit(String nom, String description, Double prix) {
        this.nom         = nom;
        this.description = description;
        this.prix        = prix;
    }
    // Getters et setters ...
}
```

Listing 2.1 – dao/Produit.java – Entité produit

2.2.2 Entité Utilisateur

```java
public class Utilisateur {
    public enum Role { ADMIN, USER }

    private String login;
    private String motDePasse;
    private Role   role;

    public boolean isAdmin() {
        return Role.ADMIN.equals(this.role);
    }
}
```

Listing 2.2 – models/Utilisateur.java – Modèle utilisateur avec rôles

2.3 Couche DAO

L'interface `ProduitDAO` définit le contrat CRUD. `ProduitImpl` l'implémente avec une `ArrayList` en mémoire, simulant une base de données.

```java
public class ProduitImpl implements ProduitDAO {
    private List<Produit> produits    = new ArrayList<>();
    private Long          compteurId  = 0L;

    public void init() {
        addProduit(new Produit("Laptop Dell XPS",
                   "Intel Core i7, 16GB RAM, 512GB SSD", 12500.0));
        addProduit(new Produit("Souris Logitech MX",
                   "Souris sans fil ergonomique", 450.0));
        // ...
    }

    @Override
    public void addProduit(Produit p) {
        compteurId++;
        p.setIdProduit(compteurId);
        produits.add(p);
    }

    @Override
    public void deleteProduit(Long id) {
        Produit aSupprimer = getProduitById(id);
        if (aSupprimer != null) produits.remove(aSupprimer);
    }

    @Override
```

```java
    public void updateProduit(Produit p) {
        for (int i = 0; i < produits.size(); i++) {
            if (produits.get(i).getIdProduit().equals(p.getIdProduit()
                )) {
                produits.get(i).setNom(p.getNom());
                produits.get(i).setDescription(p.getDescription());
                produits.get(i).setPrix(p.getPrix());
                break;
            }
        }
    }
}
```

Listing 2.3 – dao/ProduitImpl.java – Implémentation en mémoire

2.4 Couche Service

`ProduitMetierImpl` et `AuthService` sont tous deux des **Singletons**, ce qui garantit une seule instance partagée entre toutes les Servlets.

```java
public class ProduitMetierImpl implements ProduitMetier {
    private static ProduitMetierImpl instance;
    private ProduitDAO dao;

    private ProduitMetierImpl() {
        dao = new ProduitImpl();
        ((ProduitImpl) dao).init();
    }

    public static synchronized ProduitMetierImpl getInstance() {
        if (instance == null) {
            instance = new ProduitMetierImpl();
        }
        return instance;
    }
    // D l gation de toutes les op rations CRUD au DAO...
}
```

Listing 2.4 – services/ProduitMetierImpl.java – Singleton service métier

```java
public class AuthService {
    private Map<String, Utilisateur> utilisateurs = new HashMap<>();

    private AuthService() {
        utilisateurs.put("admin",
            new Utilisateur("admin", "admin123", Role.ADMIN));
        utilisateurs.put("user",
```

```
8                new Utilisateur("user",  "user123",  Role.USER));
9        }
10
11       public Utilisateur authentifier(String login, String mdp) {
12           Utilisateur u = utilisateurs.get(login);
13           if (u != null && u.getMotDePasse().equals(mdp)) return u;
14           return null;
15       }
16   }
```

Listing 2.5 – services/AuthService.java – Gestion des utilisateurs en mémoire

3. Sécurité : Authentification et gestion des rôles

3.1 Chaîne de filtres

Deux filtres sont configurés dans `web.xml` et s'exécutent **dans l'ordre** avant chaque Servlet :

1. **`AuthFilter`** — vérifie si l'utilisateur est connecté (session active).
2. **`RoleFilter`** — vérifie si l'utilisateur a le rôle `ADMIN` pour les actions sensibles.

3.1.1 Filtre d'authentification

```java
public class AuthFilter implements Filter {
    @Override
    public void doFilter(ServletRequest request, ServletResponse response,
                         FilterChain chain) throws IOException, ServletException {

        HttpServletRequest  req  = (HttpServletRequest)  request;
        HttpServletResponse resp = (HttpServletResponse) response;
        String path = req.getRequestURI().substring(
                          req.getContextPath().length());

        // URLs publiques (pas d'authentification requise)
        boolean estPublic = path.equals("/login")
                         || path.equals("/logout")
                         || path.startsWith("/css/")
                         || path.startsWith("/images/");

        if (estPublic) { chain.doFilter(request, response); return; }

        // V rification de la session
        HttpSession session    = req.getSession(false);
        boolean     estConnecte = (session != null
                    && session.getAttribute("utilisateur") != null);

        if (estConnecte) {
            chain.doFilter(request, response);
        } else {
```

```
            resp.sendRedirect(req.getContextPath() + "/login");
        }
    }
}
```

Listing 3.1 – filters/AuthFilter.java – Vérification de session

3.1.2 Filtre de rôle

```
public class RoleFilter implements Filter {
    @Override
    public void doFilter(ServletRequest request, ServletResponse response,
                         FilterChain chain) throws IOException, ServletException {

        HttpServletRequest  req  = (HttpServletRequest)  request;
        HttpServletResponse resp = (HttpServletResponse) response;
        String path = req.getRequestURI().substring(
                          req.getContextPath().length());

        boolean estActionAdmin = path.equals("/addProduit")
                               || path.equals("/deleteProduit")
                               || path.equals("/editProduit")
                               || path.equals("/updateProduit");

        if (!estActionAdmin) { chain.doFilter(request, response); return; }

        HttpSession  session     = req.getSession(false);
        Utilisateur  utilisateur = (session != null)
                ? (Utilisateur) session.getAttribute("utilisateur") : null;

        if (utilisateur != null && utilisateur.isAdmin()) {
            chain.doFilter(request, response);
        } else {
            // Renvoi vers la page d'erreur 403
            req.setAttribute("erreurCode",    403);
            req.setAttribute("erreurMessage",
                "Acc s refus  : droits insuffisants.");
            req.getRequestDispatcher("/WEB-INF/views/erreur.jsp")
               .forward(req, resp);
        }
    }
}
```

Listing 3.2 – filters/RoleFilter.java – Contrôle d'accès par rôle

3.2 Configuration dans web.xml

```
<!-- Filtre d'authentification : s'applique    toutes les URLs -->
<filter>
    <filter-name>AuthFilter</filter-name>
    <filter-class>filters.AuthFilter</filter-class>
</filter>
<filter-mapping>
    <filter-name>AuthFilter</filter-name>
    <url-pattern>/*</url-pattern>
</filter-mapping>

<!-- Filtre de r le : uniquement sur les actions ADMIN -->
<filter>
    <filter-name>RoleFilter</filter-name>
    <filter-class>filters.RoleFilter</filter-class>
</filter>
<filter-mapping>
    <filter-name>RoleFilter</filter-name>
    <url-pattern>/addProduit</url-pattern>
</filter-mapping>
<filter-mapping>
    <filter-name>RoleFilter</filter-name>
    <url-pattern>/deleteProduit</url-pattern>
</filter-mapping>
<!-- ... /editProduit, /updateProduit -->

<!-- Pages d'erreur HTTP -->
<error-page>
    <error-code>404</error-code>
    <location>/WEB-INF/views/erreur.jsp</location>
</error-page>
```

Listing 3.3 – web.xml – Déclaration des filtres et mappings

4. Servlets – Contrôleurs de l'application

4.1 Servlet de connexion – LoginServlet

```java
public class LoginServlet extends HttpServlet {
    private AuthService authService = AuthService.getInstance();

    @Override
    protected void doGet(HttpServletRequest req, HttpServletResponse resp)
            throws ServletException, IOException {
        // Redirection si d  j   connect
        HttpSession session = req.getSession(false);
        if (session != null && session.getAttribute("utilisateur") != null) {
            resp.sendRedirect(req.getContextPath() + "/listProduits");
            return;
        }
        req.getRequestDispatcher("/WEB-INF/views/login.jsp")
            .forward(req, resp);
    }

    @Override
    protected void doPost(HttpServletRequest req, HttpServletResponse resp)
            throws ServletException, IOException {
        String login      = req.getParameter("login");
        String motDePasse = req.getParameter("motDePasse");

        Utilisateur utilisateur =
            authService.authentifier(login.trim(), motDePasse);

        if (utilisateur != null) {
            HttpSession session = req.getSession(true);
            session.setAttribute("utilisateur", utilisateur);
            session.setMaxInactiveInterval(30 * 60); // 30 min
            resp.sendRedirect(req.getContextPath() + "/listProduits");
        } else {
            req.setAttribute("erreur", "Login ou mot de passe incorrect.");
            req.setAttribute("loginSaisi", login);
            req.getRequestDispatcher("/WEB-INF/views/login.jsp")
```

```
                .forward(req, resp);
        }
    }
}
```

Listing 4.1 – web/LoginServlet.java – Gestion de la connexion

4.2 Servlet de liste – ListProduitServlet

```
public class ListProduitServlet extends HttpServlet {
    private static final ProduitMetier metier =
        ProduitMetierImpl.getInstance();

    @Override
    protected void doGet(HttpServletRequest req, HttpServletResponse
        resp)
            throws ServletException, IOException {
        String idParam = req.getParameter("idProduit");
        List<Produit> liste;

        if (idParam != null && !idParam.trim().isEmpty()) {
            try {
                Long id    = Long.parseLong(idParam.trim());
                Produit  p = metier.getProduitById(id);
                if (p != null) {
                    liste = List.of(p);
                } else {
                    req.setAttribute("infoMessage",
                        "Aucun produit trouv  avec l'ID : " + id);
                    liste = metier.getAllProduits();
                }
            } catch (NumberFormatException e) {
                req.setAttribute("erreurMessage",
                    "L'ID doit  tre  un nombre entier valide.");
                liste = metier.getAllProduits();
            }
        } else {
            liste = metier.getAllProduits();
        }

        req.setAttribute("listeProduits", liste);
        req.getRequestDispatcher("/WEB-INF/views/index.jsp")
            .forward(req, resp);
    }
}
```

Listing 4.2 – web/ListProduitServlet.java – Affichage et recherche

4.3 Servlet d'ajout – AddProduitServlet

```java
protected void doPost(HttpServletRequest req, HttpServletResponse resp
    )
        throws ServletException, IOException {
    String nom         = req.getParameter("nom");
    String description = req.getParameter("description");
    String prixStr     = req.getParameter("prix");

    // Validation des champs obligatoires
    if (nom == null || nom.trim().isEmpty() || ...) {
        req.setAttribute("erreurMessage", "Tous les champs sont
            obligatoires.");
        req.setAttribute("listeProduits", metier.getAllProduits());
        req.getRequestDispatcher("/WEB-INF/views/index.jsp")
            .forward(req, resp);
        return;
    }

    try {
        Double prix = Double.parseDouble(prixStr.trim());
        if (prix < 0) throw new NumberFormatException("Prix negatif");

        Produit p = new Produit(nom.trim(), description.trim(), prix);
        metier.addProduit(p);
        // Pattern POST-REDIRECT-GET
        resp.sendRedirect(req.getContextPath() + "/listProduits");

    } catch (NumberFormatException e) {
        req.setAttribute("erreurMessage",
            "Le prix doit etre un nombre decimal positif.");
        // ...
    }
}
```

Listing 4.3 – web/AddProduitServlet.java – Ajout avec validation

4.4 Servlet de déconnexion – LogoutServlet

```java
protected void doGet(HttpServletRequest req, HttpServletResponse resp)
        throws ServletException, IOException {
```

```
    HttpSession session = req.getSession(false);
    if (session != null) {
        session.invalidate(); // Suppression compl te de la session
    }
    resp.sendRedirect(req.getContextPath() + "/login");
}
```

Listing 4.4 – web/LogoutServlet.java – Invalidation de session

5. Vues JSP

5.1 Page de connexion – `login.jsp`

La page de connexion est accessible sans authentification. Elle affiche un formulaire avec gestion des messages d'erreur via JSTL.

```jsp
<%@ taglib prefix="c" uri="http://java.sun.com/jsp/jstl/core" %>
<div class="login-boite">
    <%-- Affichage conditionnel du message d'erreur --%>
    <c:if test="${not empty erreur}">
        <div class="alerte alerte-erreur">${erreur}</div>
    </c:if>

    <form action="${pageContext.request.contextPath}/login" method="post">
        <input type="text"     name="login"      value="${loginSaisi}" />
        <input type="password" name="motDePasse" />
        <button type="submit">Se connecter</button>
    </form>
</div>
```

Listing 5.1 – login.jsp – Formulaire de connexion avec JSTL

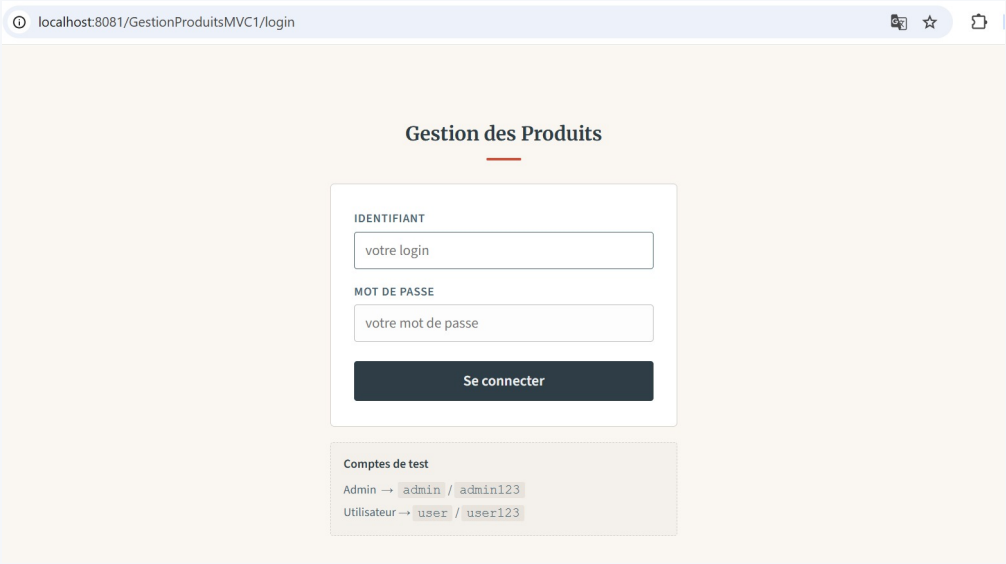

FIGURE 5.1 – Page de connexion (formulaire login/mot de passe)

5.2 Page principale – index.jsp

La vue principale est partagée entre les rôles ADMIN et USER. Les sections sensibles sont conditionnées par le rôle via JSTL.

```
<%-- Formulaire d'ajout/modification : visible uniquement pour l'ADMIN --%>
<c:if test="${sessionScope.utilisateur.role == 'ADMIN'}">
    <form action="${pageContext.request.contextPath}/
          ${not empty produitEdit ? 'updateProduit' : 'addProduit'}"
          method="post">
        <%-- Champs du formulaire --%>
        <input type="text" name="nom"         value="${produitEdit.nom}" />
        <input type="text" name="description" value="${produitEdit.description}" />
        <input type="text" name="prix"        value="${produitEdit.prix}" />
        <button type="submit">
            <c:choose>
                <c:when test="${not empty produitEdit}">Enregistrer</c:when>
                <c:otherwise>Ajouter</c:otherwise>
            </c:choose>
        </button>
    </form>
</c:if>

<%-- Tableau des produits (visible par tous) --%>
<c:forEach var="p" items="${listeProduits}">
    <tr>
        <td>${p.idProduit}</td>
        <td>${p.nom}</td>
        <td>${p.description}</td>
        <td><fmt:formatNumber value="${p.prix}"
                              minFractionDigits="2" /></td>
        <%-- Boutons d'action : ADMIN uniquement --%>
        <c:if test="${sessionScope.utilisateur.role == 'ADMIN'}">
            <td>
                <a href="${pageContext.request.contextPath}
                         /editProduit?id=${p.idProduit}">Modifier</a>
                <a href="${pageContext.request.contextPath}
                         /deleteProduit?id=${p.idProduit}"
                   onclick="return confirm('Supprimer ?');">Supprimer</a>
            </td>
        </c:if>
```

```
    </tr>
</c:forEach>
```

Listing 5.2 – index.jsp – Affichage conditionnel selon le rôle

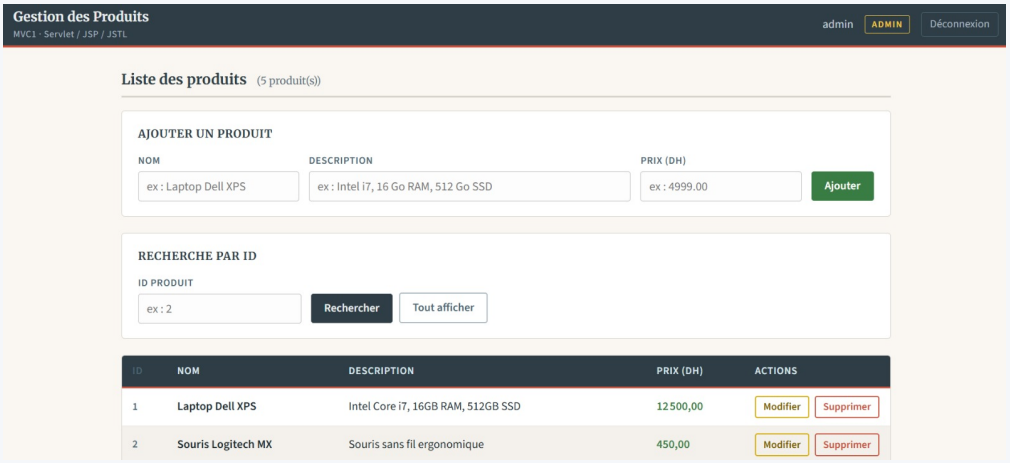

FIGURE 5.2 – Page de admin avec les privileges du role admin

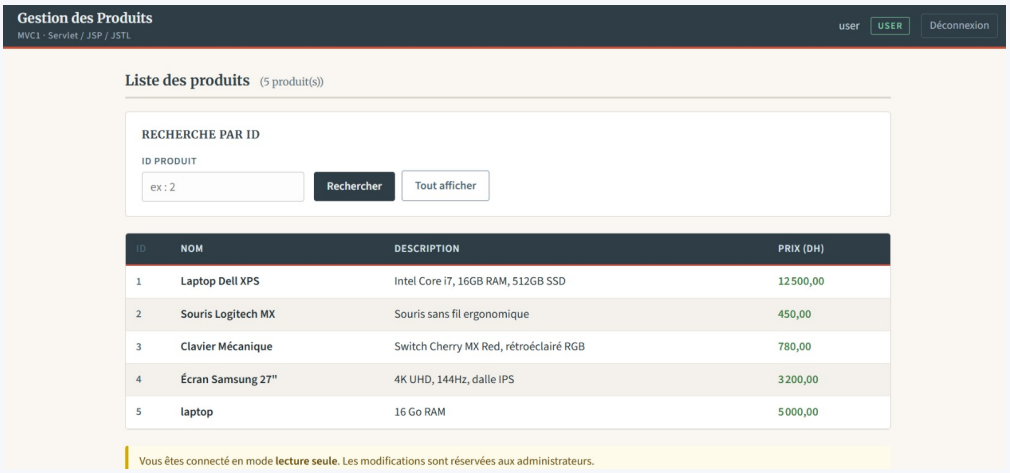

FIGURE 5.3 – Page de user standard

5.3 Page d'erreur – erreur.jsp

Une seule page gère les erreurs 403, 404 et 500. Le code d'erreur est détecté dynamiquement.

```
<%
    Integer code = (Integer) request.getAttribute("erreurCode");
    if (code == null) {
        Object sc = request.getAttribute("javax.servlet.error.
            status_code");
        code = (sc != null) ? (Integer) sc : 500;
```

```
    }
    // Message par d faut selon le code
    String msg = (String) request.getAttribute("erreurMessage");
    if (msg == null) {
        if (code == 404) msg = "La page demandee est introuvable.";
        else if (code == 403) msg = "Vous n'avez pas les droits requis
            .";
        else msg = "Une erreur inattendue s'est produite.";
    }
%>
<div class="erreur-page">
    <div class="erreur-code-grand">${codeAffiche}</div>
    <h1>
        <c:choose>
            <c:when test="${codeAffiche == 403}">Acces refuse</c:when>
            <c:when test="${codeAffiche == 404}">Page introuvable</c:
                when>
            <c:otherwise>Erreur serveur</c:otherwise>
        </c:choose>
    </h1>
    <p>${msgAffiche}</p>
    <a href="${pageContext.request.contextPath}/listProduits">
        Retour a l'accueil
    </a>
</div>
```

Listing 5.3 – erreur.jsp – Page d'erreur unifiée (403, 404, 500)

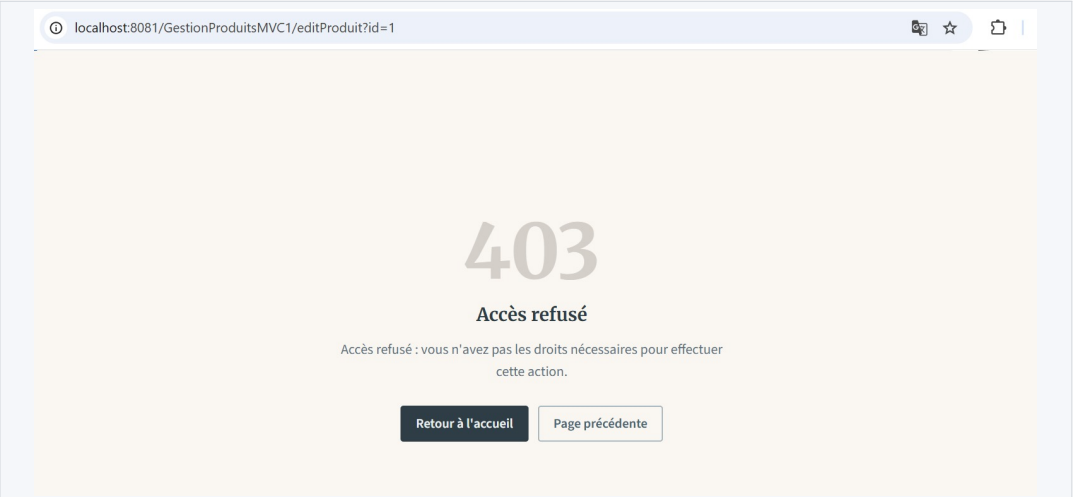

FIGURE 5.4 – Page d'erreur 403 – Accès refusé

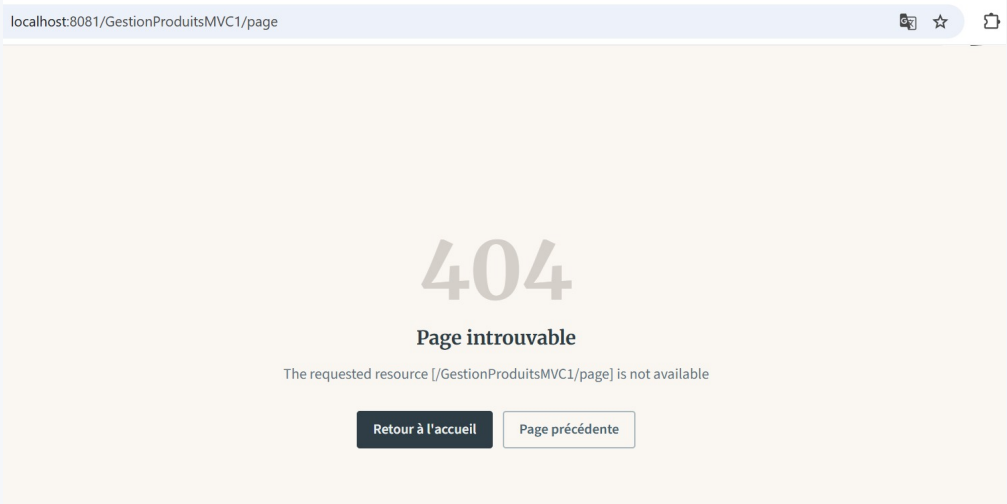

FIGURE 5.5 – Page d'erreur 404 – Page introuvable

6. Flux de travail et scénarios d'utilisation

6.1 Vue d'ensemble du flux

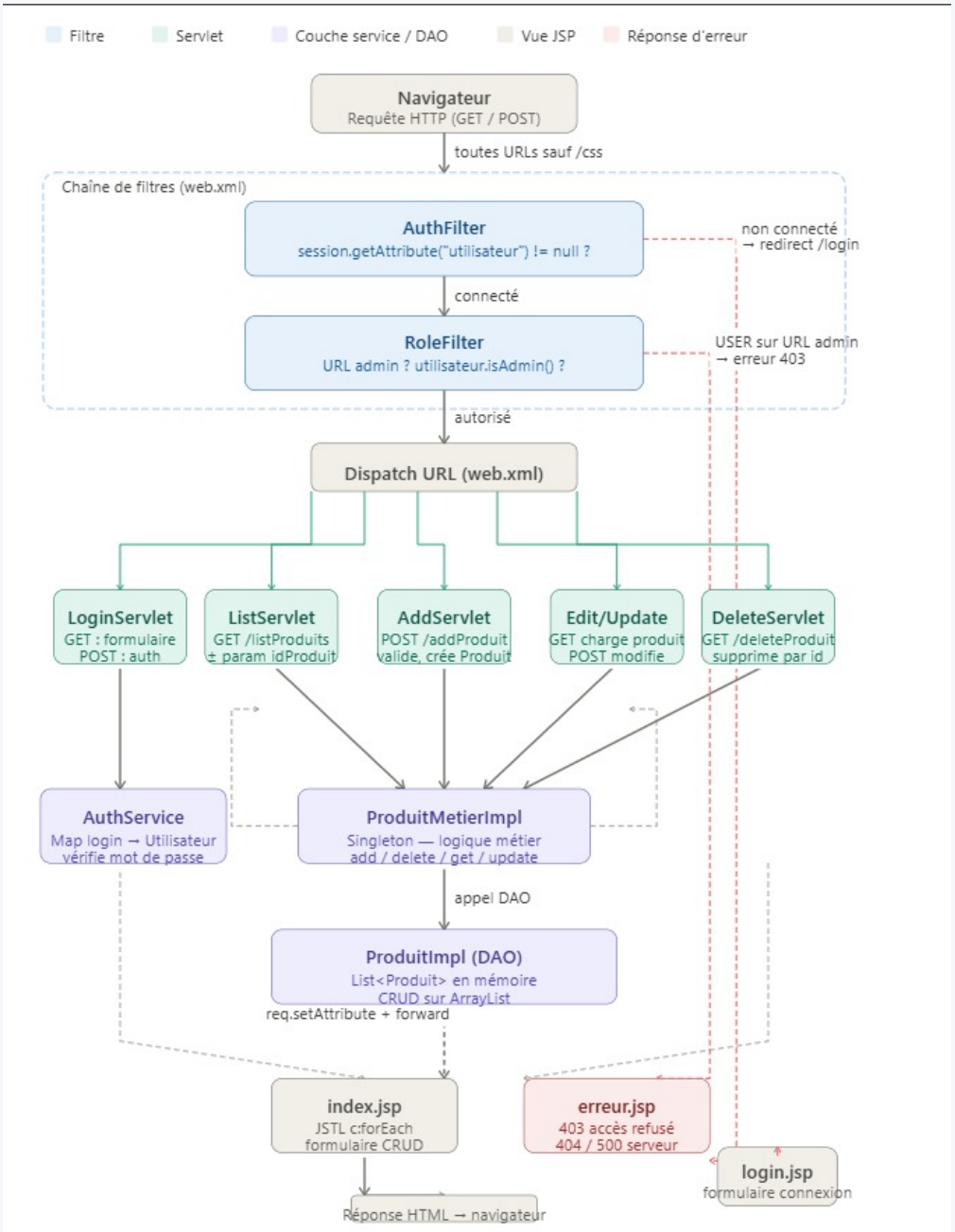

FIGURE 6.1 – Flux de traitement général de l'application

6.2 Scénario 1 – Connexion réussie (ADMIN)

Scénario 1 : Connexion en tant qu'administrateur

1. L'utilisateur accède à `/listProduits`.
2. `AuthFilter` : aucune session active → redirection vers `/login`.
3. L'utilisateur saisit `admin` / `admin123` et valide.
4. `LoginServlet.doPost()` appelle `AuthService.authentifier()`.
5. Résultat non nul : session créée avec l'objet `Utilisateur` (rôle ADMIN).
6. Redirection vers `/listProduits`.
7. `ListProduitServlet` charge tous les produits et *forward* vers `index.jsp`.
8. La vue affiche le formulaire d'ajout **et** les boutons Modifier/Supprimer.

6.3 Scénario 2 – Connexion réussie (USER)

Scénario 2 : Connexion en tant qu'utilisateur standard

1. L'utilisateur saisit `user` / `user123` et valide.
2. Session créée avec le rôle `USER`.
3. La vue `index.jsp` détecte `role != 'ADMIN'` : formulaire d'ajout masqué, boutons d'action masqués.
4. Un bandeau avertit : « *Vous êtes connecté en mode lecture seule* ».

6.4 Scénario 3 – Identifiants incorrects

Scénario 3 : Échec de connexion

1. L'utilisateur saisit un login ou mot de passe erroné.
2. `AuthService.authentifier()` retourne `null`.
3. La Servlet place un message d'erreur en attribut de requête.
4. La vue `login.jsp` réaffiche le formulaire avec le message d'erreur.
5. Le champ login conserve la valeur saisie (`${loginSaisi}`).

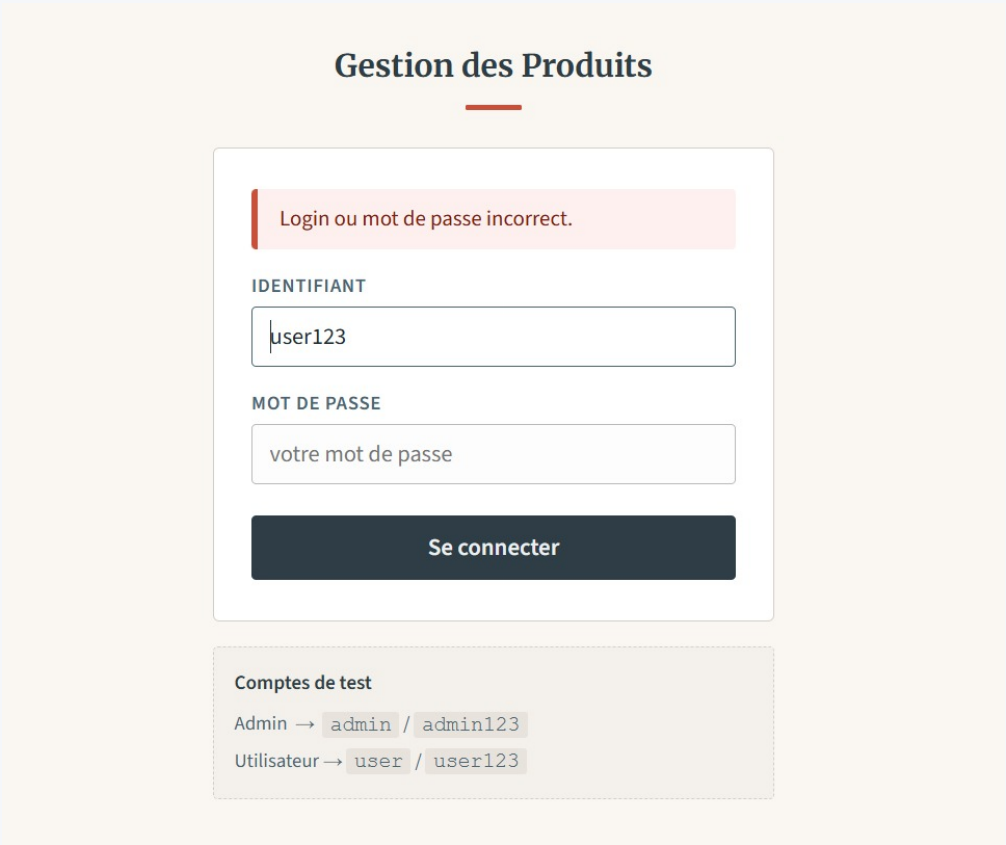

FIGURE 6.2 – Échec de connexion – message d'erreur affiché

6.5 Scénario 4 – Ajout d'un produit (ADMIN)

Scénario 4 : Ajout d'un nouveau produit

1. L'ADMIN remplit le formulaire (nom, description, prix) et valide.
2. `POST /addProduit` → `RoleFilter` laisse passer (rôle ADMIN).
3. `AddProduitServlet` valide les champs (vide ? prix négatif ?).
4. Si valide : `metier.addProduit(p)` puis redirection `POST-REDIRECT-GET`.
5. Si invalide : message d'erreur, re-affichage du formulaire.

6.6 Scénario 5 – Modification d'un produit (ADMIN)

Scénario 5 : Modification d'un produit existant

1. L'ADMIN clique sur **Modifier** (lien `/editProduit?id=X`).
2. `EditProduitServlet` charge le produit et le met en attribut `produitEdit`.
3. `index.jsp` pré-remplit le formulaire avec les données existantes.

4. L'ADMIN modifie et soumet → `POST /updateProduit`.
5. `UpdateProduitServlet` valide, appelle `metier.updateProduit(p)`.
6. Redirection vers la liste mise à jour.

6.7 Scénario 6 – Suppression d'un produit (ADMIN)

Scénario 6 : Suppression d'un produit

1. L'ADMIN clique sur **Supprimer** — une boîte de dialogue JavaScript demande confirmation.
2. Si confirmé : `GET /deleteProduit?id=X`.
3. `DeleteProduitServlet` vérifie l'existence du produit.
4. Si trouvé : `metier.deleteProduit(id)` puis redirection.
5. Si introuvable : message d'erreur affiché sur la liste.

6.8 Scénario 7 – Tentative d'accès interdit (USER → Erreur 403)

Scénario 7 : Accès refusé à une action ADMIN

1. Un utilisateur `USER` tente d'accéder directement à `/addProduit`.
2. `AuthFilter` laisse passer (session valide).
3. `RoleFilter` : `utilisateur.isAdmin()` retourne `false`.
4. Le filtre place le code 403 et un message, puis *forward* vers `erreur.jsp`.
5. La page affiche « **403 – Accès refusé** ».

6.9 Scénario 8 – URL introuvable (Erreur 404)

Scénario 8 : Page 404

1. L'utilisateur navigue vers une URL non déclarée dans `web.xml`.
2. Tomcat déclenche l'erreur 404.
3. La configuration `<error-page>` dans `web.xml` redirige vers `erreur.jsp`.
4. `erreur.jsp` récupère le code via `javax.servlet.error.status_code` et affiche « **404 – Page introuvable** ».

6.10 Scénario 9 – Déconnexion

Scénario 9 : Déconnexion de l'application

1. L'utilisateur clique sur **Déconnexion**.
2. `GET /logout` → `LogoutServlet`.
3. `session.invalidate()` supprime la session et toutes ses données.
4. Redirection vers `/login`.
5. Toute tentative de retour en arrière redirige vers le login (session invalide).

7. Tests et résultats

7.1 Tests fonctionnels

Test	Action	Résultat attendu
T01	Connexion avec `admin/admin123`	Redirection vers liste + rôle ADMIN
T02	Connexion avec `user/user123`	Redirection vers liste + mode lecture seule
T03	Connexion avec mauvais mot de passe	Message d'erreur, formulaire re-affiché
T04	Ajout d'un produit valide (ADMIN)	Produit ajouté, liste mise à jour
T05	Ajout avec champ vide	Message d'erreur, produit non créé
T06	Modification d'un produit (ADMIN)	Formulaire pré-rempli, mise à jour OK
T07	Suppression d'un produit (ADMIN)	Confirmation JS, produit supprimé
T08	USER accède à `/addProduit`	Page d'erreur 403 affichée
T09	URL inconnue	Page d'erreur 404 affichée
T10	Déconnexion	Session invalidée, retour login
T11	Recherche par ID existant	Un seul produit affiché
T12	Recherche par ID inexistant	Message info, tous les produits affichés

Table 7.1 – Plan de tests fonctionnels

7.2 Captures d'écran des résultats

FIGURE 7.1 – Test T11 – Recherche par ID réussie

FIGURE 7.2 – Test T12 – Produit non trouvé lors de la recherche

Conclusion

Ce projet a permis de mettre en pratique les concepts fondamentaux du développement web Java EE sans recours à un framework externe. L'application *GestionProduitsMVC1* illustre concrètement :

— La mise en œuvre du patron **MVC1** avec Servlets et JSP/JSTL.

— La sécurisation des accès par **filtres Servlet** (authentification et contrôle des rôles).

— L'utilisation du pattern **Singleton** pour partager l'état entre Servlets.

— La bonne pratique **POST-REDIRECT-GET** pour éviter la double soumission.

— Une gestion centralisée et homogène des erreurs HTTP.

Ce travail constitue une base solide pour aborder des projets Java EE plus avancés, notamment avec JPA/Hibernate pour la persistance en base de données, ou avec des frameworks tels que Spring MVC ou Jakarta EE pour les projets en production.

A. Comptes de démonstration

Login	Mot de passe	Rôle	Permissions
`admin`	`admin123`	ADMIN	Lister, rechercher, ajouter, modifier, supprimer des produits
`user`	`user123`	USER	Lister et rechercher des produits uniquement

Table A.1 – Comptes utilisateurs de démonstration

B. Dépendances Maven

```
<dependencies>
    <!-- API Servlet -->
    <dependency>
        <groupId>javax.servlet</groupId>
        <artifactId>javax.servlet-api</artifactId>
        <version>3.1.0</version>
        <scope>provided</scope>
    </dependency>
    <!-- JSTL -->
    <dependency>
        <groupId>javax.servlet</groupId>
        <artifactId>jstl</artifactId>
        <version>1.2</version>
    </dependency>
</dependencies>
```

Listing B.1 – pom.xml – Dépendances principales